OS ORANGE PROBLEM

ANUDEEPAK ANPURAJA
PES2UG24CS905

Phase 1:

```
make: *** [Makefile:22: test_tree] Error 1
seed@seedat905:~/PES2UG24CS905-PES-VCS$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bb
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
seed@seedat905:~/PES2UG24CS905-PES-VCS$
```

```
seed@seedat905:~/PES2UG24CS905-PES-VCS$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Phase 2:

```
seed@seedat905:~/PES2UG24CS905-PES-VCS$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
seed@seedat905:~/PES2UG24CS905-PES-VCS$ find .pes/objects -type f
.pes/objects/13/b7e821533d3fe3728a3c4560606a65aab99f4390b9df0714f9075c0ef4c2d6
```

```
seed@seedat905:~/PES2UG24CS905-PES-VCS$ xxd .pes/objects/13/b7e821533d3fe3728a3c
4560606a65aab99f4390b9df0714f9075c0ef4c2d6 | head -n 20
00000000: 626c 6f62 2031 3300 7465 7374 2063 6f6e  blob 13.test con
00000010: 7465 6e74 0a                             tent.
```

Phase 3:

```
seed@seedat905:~/PES2UG24CS905-PES-VCS$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.h
  untracked:  test_tree.c
  untracked:  test_objects
  untracked:  tree.c
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  pes.c
  untracked:  index.c
  untracked:  test_sequence.sh
  untracked:  test_tree
  untracked:  test_objects.c
  untracked:  README.md
  untracked:  Makefile
  untracked:  commit.c
```

```
seed@seedat905:~/PES2UG24CS905-PES-VCS$ cat .pes/index
100644 e67ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc 17764822
58 8 bye.txt
100644 05c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43 17764822
58 12 hello.txt
```

Phase 4:

```
./pes log
Initialized empty PES repository in .pes/
Committed: 20315531d513... Initial commit
Committed: 7fa30d512649... Add world
Committed: 556551ef1570... Add farewell
commit 556551ef157085755a6da049fa658b67d882194458c13c620c6b87a89b0b6b30
Author: PES User <pes@localhost>
Date:   1776482258

    Add farewell

commit 7fa30d512649032b5f6db8cc44bd10fb60168ab26813eeb098c10023dd08a0e8
Author: PES User <pes@localhost>
Date:   1776482258

    Add worldell

commit 20315531d51361409281170397e0e07e4db05745444c603b398ffee60fdf9412
Author: PES User <pes@localhost>
Date:   1776482258

    Initial commit
```

```
seed@seedat905:~/PES2UG24CS905-PES-VCS$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/20/315531d51361409281170397e0e07e4db05745444c603b398ffee60fdf9412
.pes/objects/3b/c19934c2ccc37b9582dd42105ca4a1df8e197f51c99a3ef71628231437fdbf
.pes/objects/4c/bf11526d072958e7d22603b6d047de827b9ed8750714cf881c0e9b6fde1d9e
.pes/objects/4d/a3c7bd705e761426425b9e3f3d98e4f2167bfbc517d0b3e021ab5b3e1243c3
.pes/objects/55/6551ef157085755a6da049fa658b67d882194458c13c620c6b87a89b0b6b30
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/70/ce788af6dc32b81308b67dcd6bd9bcdac9d4f25e72475d672d074f9e2acd70
.pes/objects/76/ddd6300e86a26597b8c7591755ea4945d5753d34b8e2bcc2a58a3a4176db3b
.pes/objects/7f/a30d512649032b5f6db8cc44bd10fb60168ab26813eeb098c10023dd08a0e8
.pes/objects/91/4ed0e34392d73a26b23baccf69b684927551d31f123a15a021a7d6f854056a
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/a7/06972e3702b2970cb301c95849fe9b33ab8c2192f373017ac97d9a4c25ecfa
.pes/objects/de/579e7a4752382545a40b76a568dfa5a1ef7ae48953ae3210105d711d54f945
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ed/ea55bdc1817fc3576b47cf0a87cff50e527908bd98fa521857bf125ad12720
.pes/refs/heads/main
```

```
seed@seedat905:~/PES2UG24CS905-PES-VCS$ cat .pes/refs/heads/main
556551ef157085755a6da049fa658b67d882194458c13c620c6b87a89b0b6b30
seed@seedat905:~/PES2UG24CS905-PES-VCS$ cat .pes/HEAD
ref: refs/heads/main
seed@seedat905:~/PES2UG24CS905-PES-VCS$
```

Phase 5&6:

Q5.1: Implementing Checkout

To implement pes checkout <branch>, the system must update the HEAD file to point to the new branch reference (e.g., ref: refs/heads/dev). Crucially, the Index must be overwritten with the tree structure of the target commit so the staging area matches the new branch. The Working Directory then undergoes a "sync" where files are added, removed, or modified to

match that index. This operation is complex because it is destructive; the system must verify that it won't overwrite uncommitted "dirty" files, requiring a three-way comparison between the current commit, the index, and the target branch.

Q5.2: Detecting a "Dirty" Directory
A "dirty" conflict is detected by comparing the Index against the Object Store. First, the system checks the index entry for a file against that same file in the Current HEAD’s tree; if the hashes differ, the file has uncommitted changes. The system then compares the index hash to the Target Branch's tree. If the target branch requires a version of the file that differs from what is currently in the index, the checkout is refused. This logic ensures that unique data sitting in the index isn't permanently lost during the branch switch.

Q5.3: Detached HEAD and Recovery
In a "Detached HEAD" state, making a commit creates a new Commit Object and updates the .pes/HEAD file with that specific hash instead of a branch name. These commits are "homeless" because no branch reference points to them. If the user switches away, the pointer is lost, making the commits invisible to standard logs. To recover them, a user must use a Reflog (a history of HEAD's movements) to find the lost hash, or manually scan the object store for "orphan" commits that have no children, then point a new branch at that hash to "anchor" them.

Q5.1: Implementing Checkout

To implement pes checkout <branch>, the system must update the HEAD file to point to the new branch reference (e.g., ref: refs/heads/dev). Crucially, the Index must be overwritten with the tree structure of the target commit so the staging area matches the new branch. The Working Directory then undergoes a "sync" where files are added, removed, or modified to match that index. This operation is complex because it is destructive; the system must verify that it won't overwrite uncommitted "dirty" files, requiring a three-way comparison between the current commit, the index, and the target branch.
Q5.2: Detecting a "Dirty" Directory

A "dirty" conflict is detected by comparing the Index against the Object Store. First, the system checks the index entry for a file against that same file in the Current HEAD’s tree; if the hashes differ, the file has uncommitted changes. The system then compares the index hash to the Target Branch's tree. If the target branch requires a version of the file that differs from what is currently in the index, the checkout is refused. This logic ensures that unique data sitting in the index isn't permanently lost during the branch switch.
Q5.3: Detached HEAD and Recovery

In a "Detached HEAD" state, making a commit creates a new Commit Object and updates the .pes/HEAD file with that specific hash instead of a branch name. These commits are "homeless" because no branch reference points to them. If the user switches away, the pointer is lost, making the commits invisible to standard logs. To recover them, a user must use a Reflog (a history of HEAD's movements) to find the lost hash, or manually scan the object store for "orphan" commits that have no children, then point a new branch at that hash to "anchor" them.

Q6.1: Garbage Collection and Reachability

To clean the object store, you would implement a mark and sweep algorithm. The process begins with a mark phase, starting at all repository roots—specifically every branch in the refs directory, the HEAD pointer, and any tags. The algorithm recursively traverses every commit, tree, and sub-tree, adding their hashes to a hash set or a bitset to track them efficiently. Once all reachable paths are mapped, a sweep phase begins where the system iterates through every physical file in the objects directory. Any file whose hash is not present in the tracking data structure is considered unreachable and is deleted. In a repository with 100,000 commits and 50 branches, the system would likely need to visit between one million and two million objects, depending on how much history is shared between branches.

Q6.2: Concurrent Dangers and Race Conditions

Running garbage collection concurrently with a commit is dangerous because it creates a high risk of repository corruption via a race condition. For example, while a commit operation is in progress, it might write a new blob to the object store before the final commit object is linked to a branch. If the garbage collector runs at that exact moment, it will see the new blob as unreachable because no branch points to it yet, and delete it. When the commit process finally tries to finalize the link, the referenced object is gone. To prevent this, systems like Git implement a grace period where the garbage collector only deletes unreachable objects that are older than a specific timestamp, such as two weeks, ensuring that fresh objects created by active processes are protected.

Github Link: https://github.com/anudeepak772/PES2UG24CS905-PES-VCS